COMP101
Introduction to Programming
2025-26

Lecture-04

String Input and Output (I/O)

Variables

Assignment

How a Program Works: Input-Process-Output

- All programs consist of lines of code ('instructions' or 'statements')
  - basically work using three parts: Input – Process – Output

Input	Process	Output
Data capture for use by the program. Data comes via keyboard, file, download, signal from a device, scanning etc. COMP101 only uses keyboard input. Once captured, data can be processed into useful information.	The lines of code to manipulate input data. Done using a combination of three 'constructs': Sequence Selection Iteration Once processed the information can be output.	Information output to screen, audio, another machine, printer etc. COMP101 outputs directly to the screen of the user.

Points to remember for COMP101

- All coding is done in IDLE
  - Don’t submit any work written in the shell
- i/o is ‘standard i/o’ – manual input
  - Input via keyboard
  - Output to VDU
- No scope for automatic input (pointers, touch etc)
  - mouse, scanner, graphic tablet, barcode, joystick, camera, light pen, touchscreen, file, sensors, microphone …

How a Program Works: Constructs

- Processing data using combinations of three ‘constructs’
  - Sequence and/or Selection and/or Iteration
- Sequence
  - ALL lines of code in the program WILL execute once and once only
    - one line at a time and in the order in which they have been typed by the programmer
- Selection (‘if’)
  - SOME lines of code will execute, some will NOT execute
- Iteration (‘loops’)
  - SOME lines of code will execute MORE than once

input(): terminology

- data (singular datum) is input to a program via keyboard, scan, file etc
- data has value (3, 3.142. ‘cat’, Y/N etc)
- value has a data type
- data type
  - 3 is numeric and can be further defined as ‘numeric integer’
  - 3.142 is numeric and can be further defined as ‘numeric float’
  - ‘cat’ is string (note apostrophes) comprising three char data types
  - Y/N is Boolean

Some data types (longint, double, char) are not used in Python

Input a string: assignment statement

- We use ‘statements’ – single lines of code to make things work
- ‘Assignment statement’ (‘variable declaration’) captures keyboard i/p
- Three elements in the assignment statement - in this order
  - a single name invented by programmer — ‘variable name’ or ‘var’
  - = — ‘assignment operator’
  - a single function name — ‘input () function’

myName = input(‘Enter your name: ‘)

‘assignment operator’ – not the ‘equals sign’
It follows the var name

Input() is the function name and has parentheses
‘Enter your name: ‘ is the ‘argument’ to the function
it goes in the parentheses – here it is an input prompt

Input a string: assignment

```
name        =    input('Enter employee name: ')
job         =    input('Enter job title: ')
id          =    input('Enter employee id: ')
telephone   =    input('Enter employee telephone: ')
salary      =    input('Enter salary: ')
payScale    =    input('Enter payscale: ')
```

Whatever the user types in will be assigned to the var name

Python does not use a 'statement de-limiter' to end a line of code – just press the return key to end a line of code

The above is a 'sequence construct' consisting of six lines of code (six 'statements')

All six statements will execute once only in the order they are coded

Input a string: assignment

variable_name (var)	assignment	input () function
myName	=	input('Enter your name: ')

- user inputs word cat: data has a value of 'cat'
  - cat is assigned to the var
- value 'cat' has a data type: 'cat' is made up of three characters
  - it is a string and it makes myName a string var

- Anything in apostrophes is a string

  Problem: The input() function assigns everything as a string
- if input data is 1234, myName has the string value '1234' assigned

Input: Assignment – other terms used

myName = input('Enter your name: ') test

'variable name' or 'var' or 'symbolic name' or 'identifier' or 'programmer defined name'.

Called a variable because it can hold different values depending on the input.

Can be any word – but there are some naming conventions to follow for what can and can't be used.

'Assignment operator'
Assigns user input to a variable name:
myName = test
Assign string 'test' to var myName

myName = 5
Assign string '5' to var myName

Note: == is the equals sign used for maths

input ()
Use 'input' followed by parentheses

In the parentheses use quotes – single or double

Inside the quotes prompt to the user to input something.
Keep it simple for the user.

Called a function as it only works when has parentheses after it – something must be put into the parentheses.

Input: Why don’t we declare variables and data types before we use them?

- Python uses ‘Implicit declaration’ (‘loosely typed’, ‘dynamic typed’)
  - The interpreter recognises the data type (string, integer, float, Boolean) by what the user types in and implicitly assigns this value and type to the var
  - It means we don’t have to declare vars and data types etc before we code them in the body of the program. We can type them in as an assignment statement and Python ‘knows’ the data type.
- ‘Explicit Declaration’ (‘strongly typed’, ‘static typed’)
  - Some languages can’t use variables unless they are ‘declared’ with a name and a type at the top of the code, usually before the input is asked for
  - Python CAN use ‘explicit declaration’ – but this is not recommended

Input - Explicit Declaration: Not usual in Python

- Declaration part

#variables

age = int

depth = float

myName = string

too_old = boolean

In non-Python code, vars, constants and functions must be declared together with their type ('declarative') before they can be used in the main body of the program.
An error occurs if we try to use a var etc that has not been declared

#constants

pi = 3.142

Constants have values assigned that never change all through the program.
We get an error if we try to change it

The general structure of a non-Python program is usually:

Heading
name of program etc

Declarations
vars, (no constants in Python)

Body
where the working code is

Calls
where we make the code work in the order in which we want

Input – Python CAN use explicit assignment – but not usual practice: Don't do it

- At the top of the Python program before any other coding

```
x = 3          #integer variable
pi = 3.142     #float variable
y = 'c'        #string variable (no 'char' type in Python)
```

Python will work out the data type from what is typed in, so we don't need to include the type in the declaration

- Python allows multiple assignments – not recommended

```
#avoid doing this
x = y = z = 0
x, y, z = 3, "c", 3.142
```

Anything starting with # will turn red.
It is a comment and is ignored by the program

Input - Python data types not supported

- char

In other languages, ‘char’ is used for single characters and ‘string’ for words.
Python treats everything as a string, even single characters

- longint
- double

In other languages, these extend the range of numbers – makes them bigger or smaller.
Python dynamically allocates space as needed to deal with the size of the number

- Constants

```
PI = 3.142    # note capitalisation
```

  - Constants are not supposed to be altered in execution of a program
  - But Python treats implicit declarations as variables – and variables can be altered in execution of code
  - Current practice is to name in CAPITALS – and remember if it’s in capitals, don’t change it!

Output: string

- Output in Python
  - We use print() – ‘print function’
  - Called a function because it has parentheses after it – this means something must be put into the parentheses to output it onto the screen

print(‘Hello World’) #use single or double quotes – just be consistent

- This is a ‘string literal’ - no user input here – it’s been typed in already
- Always outputs the same content (screen output is the default)
- 11 strings (no char type in Python) in Hello World (the ‘output string’)
  - everything is a string in Python, even single characters including the space

Input and Output: string

- We usually prompt the user for input and then perform output on it
- The output may be exactly what the user has typed in or it may be output of something different if it has been processed in between input and output

- input() will assign a value to a var using the assignment statement
- print() will output to screen the value of the var
- The name of the var in the output() statement will be the same name as used in the input() statement

Input and Output: string
Input a string and then output it

```
#prompt for input – user types in the word red
colour = input("Enter a colour: ") red
#nothing happens until <return> is pressed:  Remember to press it!
#programmer-defined var 'colour' has value 'red' assigned to it as a string data type
#print out the input, use the variable name with no quotes
print (colour)
red
```

Python is case-sensitive – spell the var name exactly as it was assigned
'Colour' is a different var name, 'color' is a different var name;

Don't put quotes in the print statement: print ('colour')
This makes it a string literal and will output the string 'colour' instead of the value 'red' which is assigned to var 'colour'

Input and Output: string
Can't see the output?

- Sometimes the program appears not to have worked
  - screen appears blank after we run the program
  - What has happened is the output has worked
    - but the screen refreshes before you get a chance to read it

- Add an input() statement to 'hold' the screen:

```
colour = input("Enter a colour: ") red     #press return after typing this in
print (colour)                             #output to screen the value held in var 'colour'
input()                                    #program now waits for you to press return
red                                        #expected output
```

Input and Output: string
This time we enter a number

```
#prompt for a number
age = input("Enter your age: ") 21        #this is string '21'
print (age)                               #output the input - same var name with no quotes
21
```

input() function defaults to string entry

- var age is assigned string '21'
  - If we do maths on it – we get an error

Input and Output: string

user_name = input('Enter a name: ') Yan

num = input ("Enter an age: ") 21

Quotes in the prompt can be single quotes or double quotes – don't mix them in the parentheses and keep to one style when coding

The colon and the space after it are cosmetic – but aid usability

- Execution halts until user enters something and presses return

- Use sensible instructions - make sure users enter what you expect
  - Enter a name: Giraffe
  - Enter an age: Jurassic

Variable name guidelines
REMEMBER – Python is case-sensitive

We can use the following character sets for variable names:

- Letters ‘A’ - ’Z’, ‘a’ - ’z’
- Numbers ‘0 - 9’ #can’t start a var name with a number
- Underscore ‘_’

- When thinking up a var name:
  - Avoid using a name similar to an existing variable name
  - Use sensible names that reflect the content
  - Usually maximum of 15 characters (but not critical)

Variable name guidelines
House Styles and notation

Notation-1 (more than one word for a single var name)

- Begin each word with lower-case letter and join them up using the underscore for easier readability

  - family_name
  - given_name
  - next_of_kin

Don’t mix and match cases and styles inside the same program. Choose a style and keep to it in the program – makes debugging and readability easier

Variable name guidelines
House Styles and notation

Notation-2 (more than one word for a single var name)

- lowerCamelCase
  - Begin with lower-case
  - Next word begins with upper-case
    - finalPayment
    - nextOfKin

- MixedCase
  - Begin with upper case
  - Next word begins with upper-case
    - FinalPayment
    - NextOfKin

These notations are less conventional
Don't 'mix and match' notations
Of the two, lowerCamelCase is more popular

Technically, a string is uppercase if all cased characters in the string are uppercase and there is at least one cased character in the string

Variable name guidelines
Keep things simple

- Age_now age_now ageNow (same words but three different var names)

- f_name
  - is f_name meant to be: first_name, family_name?

- g_name
  - is g_name meant to be: grandparent_name, given_name, gilt_name?

- nok
  - is nok meant to be: no okay, no_one_known, next_of_kin, name of king?

- Think about the user who is using your program and the domain in which it operates

Variable name guidelines
Keep things simple

a = Enter length:

b = Enter depth:

C = Enter width:

Simplify this:

Use vars of length, depth, width for readability and ease of debugging;

C (by convention) is uppercase and to be treated as a constant – i.e. something not to be changed

Variable name guidelines- what we can't use: Reserved Words, Arithmetic operators, Symbols, Spaces

- and, as , assert
- break
- class, continue
- def, del
- elif, else, except
- False, finally, for, from
- global
- if, import, in, is
- lambda
- none, nonlocal, not
- or
- pass
- raise, return
- True, try
- while, with
- yield

At the interactive prompt in the shell window, type:
>>>help()
At the help prompt type:
help> keywords

Reserved words ('keywords') have a special meaning in Python: They give an error if they are not used for what they are intended for in Python syntax

Python: Programming Style

- Python official House Style (taken from PEP008 – don't learn it!)
  - Okay to vary the style a bit (no penalty) – keep it in mind when coding
- Use quotes (single, double or triple) for clarity and reading of code
- Keep it simple for debugging and readability
- Use #comments appropriately
  - Don't over-comment – use them to explain tricky bits of code

input control

```
name        =    input('Enter employee name: ')
job         =    input('Enter job title: ')
id          =    input('Enter employee id: ')
telephone   =    input('Enter employee telephone: ')
salary      =    input('Enter salary: ')
payscale    =    input('Enter payscale: ')
```

Whatever is in the parentheses and in quotes will output to the screen

This is a 'sequence' – all lines execute in the order written

We save it as a .py file and then run ('execute') it

Close the shell window and we return to the IDLE editor

Execution takes place in the shell window

```
Python 3.7.5 (tags/v3.7.5:5c02a39a0b, Oct 15 2019, 00:11:34) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: D:/COMP101_2023-24/Week-01/Lectures/i_o control.py ==========
Enter employee name: test
Enter job title: test1
Enter employee id: test2
Enter employee telephone: test3
Enter salary: test4
Enter payscale: test5
>>>
```

The input() function displays the message ('prompt') in the parentheses and execution halts until user types something in followed by return.
Whatever is typed in is assigned to the var name.

It is possible for the user to press return without typing anything in – in which case the var name is assigned a space i.e. nothing in it

input errors

```
name        =       input(‘Enter employee name: “)
                    #syntax error:  EOL while scanning string literal
```

a string literal is something typed in by the programmer

```
name = input ('Enter employee name: '
job = input ('Enter job title: ')
                    #syntax error:  invalid syntax
```

The first word (job) in the second line of code becomes highlighted in red.
It gives the impression the error is on this line but it is actually on the line above it – no closing parenthesis.
The interpreter converts the code one line at a time (unlike a compiler that converts every line of code and presents a list of errors at the end).
The first line has an error in it but as the interpreter works on a ‘read ahead’ principle it has begun reading the second line when it realises there is an error on the first line.

input errors

```
name        =       input ‘Enter employee name: ‘)
                    #syntax error:  invalid syntax
```

The error is at the start – missing an opening parenthesis:
The interpreter works on a ‘read ahead’ principle and is looking for an opening parenthesis – it raises an error when it reads in the closing parenthesis, hence the apostrophe is highlighted in red when the closing parenthesis is read in on a ‘read-ahead’ principle

Syntax

- The correct words in the correct order
  - Rules = Subject + Verb + Object

- “The cat sat on the mat” – statement can be ‘parsed’ for correctness
  - parsing
    - Article noun verb preposition article noun

      The cat sat on the mat

      The statement is ‘well formed’ – it is syntactically correct and makes sense

Syntax

“The on sat the mat cat”

- Syntactically incorrect – does not follow the rules for a ‘well formed’ statement

```
name  =  input(‘Enter employee name: “)
```

#syntax error: EOL while scanning string literal

Syntactically incorrect – does not follow the rules for ‘well formed’

```
name = input ('Enter employee name: '
job = input ('Enter job title: ')
```

#syntax error: invalid syntax

Syntactically incorrect – does not follow the rules for ‘well formed’

o/p control

```
Enter employee name: t1
Enter job title: t2
Enter employee id: t3
Enter employee telephone: t4
Enter salary: t5
Enter payscale: t6
t1
t2
t3
t4
t5
t6
>>>
```

```
name = input ('Enter employee name: ')
job = input ('Enter job title: ')
id = input ('Enter employee id: ')
telephone = input ('Enter employee telephone: ')
salary = input ('Enter salary: ')
payscale = input ('Enter payscale: ')

print (name)
print (job)
print (id)
print (telephone)
print (salary)
print (payscale)
```

var names must be exactly the same as used in the input() function.
No apostrophes and are case-sensitive

Remember the parentheses or we get a syntax error

o/p control

```
Enter employee name: t1
Enter job title: t2
Enter employee id: t3
Enter employee telephone: t4
Enter salary: t5
Enter payscale: t6
t1
t2
t3
t4
t5
T6
name
>>>
```

```
name = input ('Enter employee name: ')
job = input ('Enter job title: ')
id = input ('Enter employee id: ')
telephone = input ('Enter employee telephone: ')
salary = input ('Enter salary: ')
payscale = input ('Enter payscale: ')

print (name)
print (job)
print (id)
print (telephone)
print (salary)
print (payscale)
print (‘name’)
```

Note the last o/p line – it uses apostrophes – this is now a ‘string literal’ and will always print ‘name’ on screen

i/o control – think about the user

- Instruct user to input red item from list and ‘echo’ the input
- pink
- purple
- blue
- red
- orange
- green